

- [Connectly API Technical Documentation](#)
 - [System Architecture Overview](#)
 - [Table of Contents](#)
 - [System Architecture](#)
 - [Core Layers](#)
 - [Data Models](#)
 - [Database Schema Optimizations](#)
 - [Authentication and Authorization](#)
 - [Authentication Methods](#)
 - [Authorization Process](#)
 - [CRUD Operations Flow](#)
 - [User CRUD Flow](#)
 - [Post CRUD Flow](#)
 - [Comment, Like, and Follow Operations](#)
 - [Data Flow Patterns](#)
 - [Feed Generation Data Flow](#)
 - [Social Interaction Data Flow](#)
 - [Error Handling](#)
 - [Error Handling Strategy](#)
 - [Performance Monitoring](#)
 - [Access Control System](#)
 - [Access Control Components](#)
 - [Performance Considerations](#)

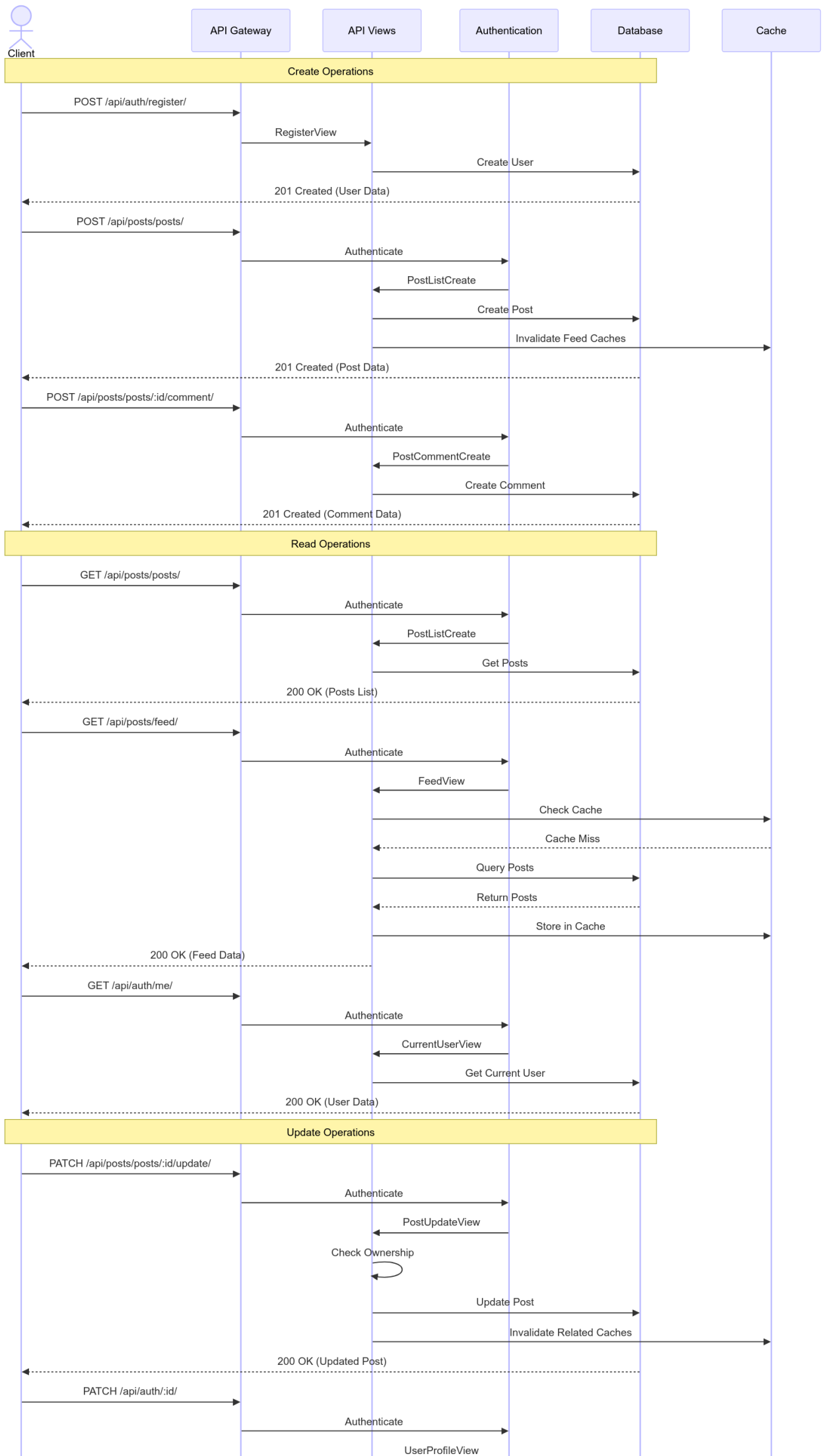
Connectly API Technical Documentation

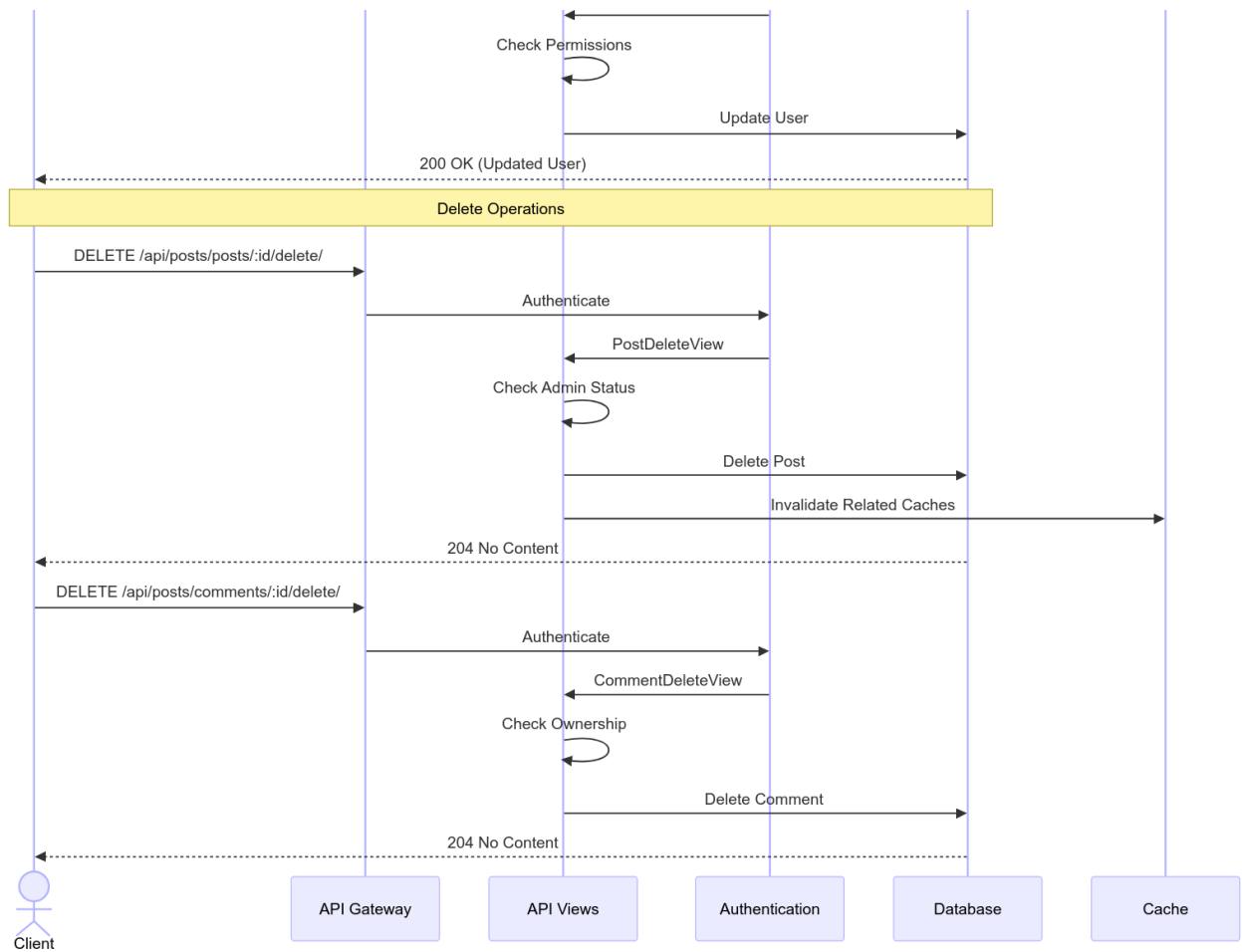
System Architecture Overview

The Connectly API is a Django-based RESTful application designed to provide social media platform functionality. This documentation covers the architecture, data models, flow patterns, and security mechanisms implemented in the system.

Table of Contents

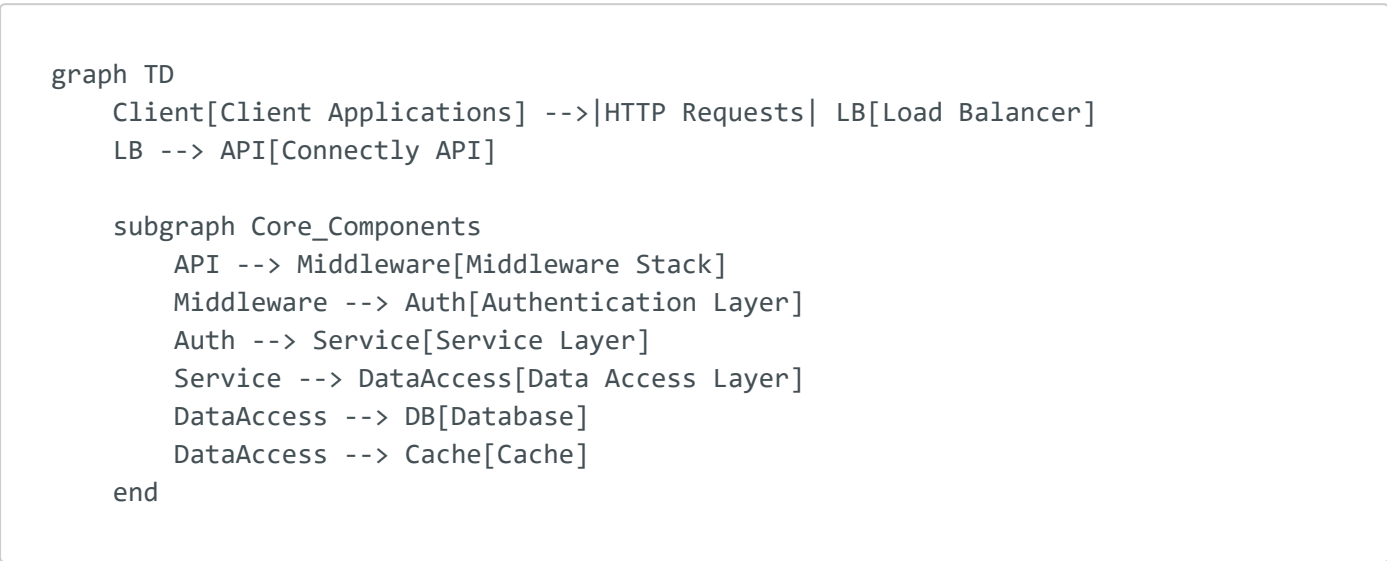
1. System Architecture
 2. Data Models
 3. Authentication and Authorization
 4. CRUD Operations Flow
 5. Data Flow Patterns
 6. Error Handling
 7. Access Control System
 8. Performance Considerations
-





System Architecture

The Connectly API follows a layered architecture pattern with distinct responsibility separation:



Core Layers

- 1. Middleware Layer:** Handles cross-cutting concerns like CSRF protection, role-based access control, and performance tracking.

2. **Authentication Layer:** Provides multiple authentication methods (JWT, Session, OAuth).
 3. **Service Layer:** Implements business logic for users, posts, comments, likes, and follows.
 4. **Data Access Layer:** Manages database interactions through Django models and serializers.
 5. **Caching Layer:** Optimizes performance for frequently accessed data.
-

Data Models

The system uses five primary entities with well-defined relationships:

```
erDiagram
    CustomUser ||--o{ Post : "creates"
    CustomUser ||--o{ Comment : "writes"
    CustomUser ||--o{ Like : "performs"
    CustomUser ||--o{ Follow : "initiates as follower"
    CustomUser ||--o{ Follow : "receives as followed"
    Post ||--o{ Comment : "contains"
    Post ||--o{ Like : "receives"
    Comment ||--o{ Comment : "parent of"

    CustomUser {
        int id PK
        string username
        string email
        string password
        string first_name
        string last_name
        string role
        bool is_active
        bool is_staff
        bool is_superuser
        datetime date_joined
        datetime last_login
    }

    Post {
        int id PK
        int author_id FK
        text content
        string privacy
        datetime created_at
    }

    Comment {
        int id PK
        int author_id FK
```

```
    int post_id FK
    int parent_id FK "nullable"
    text content
    datetime created_at
}
```

```
Like {
    int id PK
    int user_id FK
    int post_id FK
    datetime created_at
}
```

```
Follow {
    int id PK
    int follower_id FK
    int followed_id FK
    datetime created_at
}
```

Database Schema Optimizations

The database schema includes several optimizations:

1. **Indexing Strategy:** Indexes on foreign keys and frequently queried fields
2. **Composite Constraints:** Unique constraints on (user, post) for likes and (follower, followed) for follows
3. **Self-referential Relationships:** Comments support threaded discussions through parent-child relationships

Authentication and Authorization

The authentication and authorization system supports multiple methods and implements a robust role-based access control system:

```
flowchart TD
    Client((Client))

    subgraph "Authentication Methods"
        SessionAuth[Session Authentication]
        JWTAuth[JWT Authentication]
        GoogleAuth[Google OAuth]
    end
    end
```

```
subgraph "Authentication Process"
    Credentials{Validate Credentials}
    SessionCreate[Create Session]
    TokenCreate[Generate JWT Tokens]
    GoogleValidate[Validate Google Token]
    AuthSuccess[Authentication Success]
    AuthFailure[Authentication Failed]
end
```

```
subgraph "Authorization Process"
    AuthCheck{Check Authentication}
    RoleCheck{Check User Role}
    OwnerCheck{Check Resource Ownership}
    PrivacyCheck{Check Privacy Settings}
    AccessGranted[Access Granted]
    AccessDenied[Access Denied - 401/403]
end
```

%% Authentication Flows

```
Client -->|Username/Password| SessionAuth
```

```
Client -->|Username/Password| JWTAuth
```

```
Client -->|Google Token| GoogleAuth
```

```
SessionAuth --> Credentials
```

```
JWTAuth --> Credentials
```

```
GoogleAuth --> GoogleValidate
```

```
GoogleValidate -->|Valid Token| AuthSuccess
```

```
GoogleValidate -->|Invalid Token| AuthFailure
```

```
Credentials -->|Valid| SessionCreate
```

```
Credentials -->|Valid| TokenCreate
```

```
Credentials -->|Invalid| AuthFailure
```

```
SessionCreate --> AuthSuccess
```

```
TokenCreate --> AuthSuccess
```

```
AuthSuccess -->|Session Cookie| Client
```

```
AuthSuccess -->|JWT Tokens| Client
```

```
AuthFailure -->|Error Response| Client
```

%% Authorization Flows

```
Client -->|API Request with Auth| AuthCheck
```

```
AuthCheck -->|Authenticated| RoleCheck
```

```
AuthCheck -->|Unauthenticated| AccessDenied
```

```
RoleCheck -->|Admin Required?| AdminCheck{Is Admin?}
```

```
RoleCheck -->|Moderator Required?| ModCheck{Is Moderator?}
```

```
RoleCheck -->|Regular User| OwnerCheck
```

```
AdminCheck -->|Yes| AccessGranted
```

```
AdminCheck -->|No| AccessDenied
```

```
ModCheck -->|Yes| AccessGranted
```

```
ModCheck -->|No| AccessDenied
```

```
OwnerCheck -->|Is Owner| AccessGranted
OwnerCheck -->|Not Owner| PrivacyCheck
```

```
PrivacyCheck -->|Public Resource| AccessGranted
PrivacyCheck -->|Private Resource| AccessDenied
```

```
AccessGranted -->|Process Request| Client
AccessDenied -->|401/403 Response| Client
```

Authentication Methods

1. **Session Authentication:** Traditional cookie-based authentication for browser clients
2. **JWT Authentication:** Stateless token-based authentication for mobile/SPA clients
3. **Google OAuth:** Social authentication with Google as identity provider

Authorization Process

1. **Role-Based Controls:** Four user roles (admin, moderator, user, guest) with hierarchical permissions
2. **Resource Ownership:** Only owners can modify their content (posts, comments)
3. **Privacy Settings:** Content can be public (visible to all) or private (visible only to creator)

CRUD Operations Flow

The API follows standard RESTful CRUD operations for all resources. Below are the flows for the main entities:

User CRUD Flow

```
flowchart TD
    Client((Client))

    %% CREATE
    Client -->|POST /api/auth/register/| RegisterAPI[RegisterView]
    RegisterAPI -->|validate| UserValidation{Valid?}
    UserValidation -->|Yes| CreateUser[Create CustomUser]
```



```

CreateUser --> DatabaseCreate[(Database)]
CreateUser --> ReturnUser[Return User Data]
ReturnUser --> Client
UserValidation -->|No| ReturnError[Return Validation Error]
ReturnError --> Client

%% READ
Client -->|GET /api/auth/me/| CurrentUserAPI[CurrentUserView]
CurrentUserAPI -->|Authentication| AuthCheck{Authenticated?}
AuthCheck -->|Yes| FetchUser[Fetch Current User]
FetchUser --> DatabaseFetch[(Database)]
FetchUser --> SerializeUser[Serialize User]
SerializeUser --> Client
AuthCheck -->|No| Auth401[Return 401 Unauthorized]
Auth401 --> Client

%% UPDATE
Client -->|PATCH /api/auth/:user_id/| UpdateAPI[UserProfileView]
UpdateAPI -->|Authentication| UpdateAuthCheck{Authenticated?}
UpdateAuthCheck -->|Yes| PermissionCheck{Owner or Admin?}
PermissionCheck -->|Yes| ValidateUpdate{Valid Data?}
ValidateUpdate -->|Yes| UpdateUser[Update User]
UpdateUser --> DatabaseUpdate[(Database)]
UpdateUser --> ReturnUpdated[Return Updated User]
ReturnUpdated --> Client

%% DELETE
Client -->|DELETE /api/auth/:user_id/delete/| DeleteAPI[UserDeleteView]
DeleteAPI -->|Authentication| DeleteAuthCheck{Authenticated?}
DeleteAuthCheck -->|Yes| AdminCheck{Is Admin?}
AdminCheck -->|Yes| DeleteUser[Delete User]
DeleteUser --> DatabaseDelete[(Database)]
DeleteUser --> Return204[Return 204 No Content]
Return204 --> Client

```

Post CRUD Flow

flowchart TD

Client((Client))

%% CREATE

```

Client -->|POST /api/posts/posts/| CreatePostAPI[PostListCreate]
CreatePostAPI -->|Authentication| PostAuthCheck{Authenticated?}
PostAuthCheck -->|Yes| ValidatePost{Valid Data?}
ValidatePost -->|Yes| CreatePost[Create Post]
CreatePost --> DatabaseCreatePost[(Database)]
CreatePost --> ClearCache[Clear User Caches]
ClearCache --> ReturnPost[Return Post Data]
ReturnPost --> Client

```

%% READ

```

Client -->|GET /api/posts/posts/| ListPostsAPI[PostListCreate]

```

```

ListPostsAPI -->|Authentication| ListPostsAuth{Authenticated?}
ListPostsAuth -->|Yes| FetchPosts[Fetch Posts]
FetchPosts --> DatabaseFetchPosts[(Database)]
FetchPosts --> PaginatePosts[Paginate Results]
PaginatePosts --> SerializePosts[Serialize Posts]
SerializePosts --> Client

%% UPDATE
Client -->|PATCH /api/posts/posts/:post_id/update/|
UpdatePostAPI[PostUpdateView]
UpdatePostAPI -->|Authentication| UpdatePostAuth{Authenticated?}
UpdatePostAuth -->|Yes| OwnerCheck{Is Owner?}
OwnerCheck -->|Yes| ValidateUpdatePost{Valid Data?}
ValidateUpdatePost -->|Yes| UpdatePost[Update Post]
UpdatePost --> DatabaseUpdatePost[(Database)]
UpdatePost --> ReturnUpdatedPost[Return Updated Post]

%% DELETE
Client -->|DELETE /api/posts/posts/:post_id/delete/|
DeletePostAPI[PostDeleteView]
DeletePostAPI -->|Authentication| DeletePostAuth{Authenticated?}
DeletePostAuth -->|Yes| AdminRoleCheck{Is Admin?}
AdminRoleCheck -->|Yes| DeletePost[Delete Post]
DeletePost --> DatabaseDeletePost[(Database)]
DeletePost --> Return204Post[Return 204 No Content]

```

Comment, Like, and Follow Operations

Similar CRUD flows are implemented for Comments, Likes, and Follows, with appropriate permission checks and cache invalidation where needed.

Data Flow Patterns

The system implements optimized data flows for various operations:

Feed Generation Data Flow

```

graph TD
    Client((Client))
    Database[(Database)]
    Cache[(Redis Cache)]

    %% Feed Flow with Caching
    Client -->|GET /api/posts/feed/| FeedView[Feed View]

```

```
FeedView -->|Build Cache Key| FeedView
FeedView -->|Check Cache| Cache

Cache -->|Cache Miss| QueryDB[Query Database]
QueryDB -->|Get Posts| Database
QueryDB -->|Privacy Filter| QueryDB
QueryDB -->|Annotate Counts| QueryDB
QueryDB -->|Paginate| QueryDB
QueryDB -->|Store Results| Cache
QueryDB -->|Return Feed Data| FeedView

Cache -->|Cache Hit| ReturnCached[Return Cached Data]
ReturnCached -->|Return Feed Data| FeedView

FeedView -->|Format Response| FeedView
FeedView -->|Add Pagination Links| FeedView
FeedView -->|Return Feed| Client
```

Social Interaction Data Flow

```
graph TD
    Client((Client))
    Database[(Database)]
    Cache[(Redis Cache)]

%% Like Flow
Client -->|POST /api/posts/posts/:id/like/| LikeView
LikeView -->|Check Exists| Database
LikeView -->|If Not Exists| CreateLike[Create Like]
CreateLike -->|Save| Database
CreateLike -->|Return 201| Client
LikeView -->|If Exists| DeleteLike[Delete Like]
DeleteLike -->|Remove| Database
DeleteLike -->|Return 204| Client

%% Follow Flow
Client -->|POST /api/posts/follow/:user_id/| FollowView
FollowView -->|Check Exists| Database
FollowView -->|If Not Exists| CreateFollow[Create Follow]
CreateFollow -->|Save| Database
CreateFollow -->|Return 201| Client
FollowView -->|If Exists| DeleteFollow[Delete Follow]
DeleteFollow -->|Remove| Database
DeleteFollow -->|Return 204| Client
```

Error Handling

The API implements a comprehensive error handling strategy:

flowchart TD

Client([Client Request]) --> Middleware[Middleware Stack]

subgraph "Request Validation"

Middleware --> AuthCheck{Authentication
Check}

AuthCheck -->|Valid Auth| PermissionCheck{Permission
Check}

AuthCheck -->|Invalid Auth| Auth401[Return 401 Unauthorized]

PermissionCheck -->|Has Permission| InputValidation{Input
Validation}

PermissionCheck -->|No Permission| Permission403[Return 403 Forbidden]

InputValidation -->|Valid Data| ProcessRequest[Process Request]

InputValidation -->|Invalid Data| Validation400[Return 400 Bad Request]

end

subgraph "Exception Handling"

ProcessRequest --> TryBlock{Try Block}

TryBlock -->|Success| SuccessResponse[Return Success Response]

TryBlock -->|Exception| ExceptionHandler{Exception
Type}

ExceptionHandler -->|ValidationError| ValidationException[Return 400 Bad Request
with Validation Errors]

ExceptionHandler -->|PermissionDenied| PermissionException[Return 403 Forbidden]

ExceptionHandler -->|ObjectDoesNotExist| NotFoundException[Return 404 Not Found]

ExceptionHandler -->|Other Exceptions| ServerException[Return 500 Internal Server Error]

end

subgraph "Middleware Error Handling"

Middleware -->|CSRF Error| CSRFFailure[Return 403 CSRF Failure]

Middleware -->|Rate Limit Exceeded| RateLimitExceeded[Return 429 Too Many Requests]

Middleware -->|Role Permission Error| RoleMiddlewareError[Return 403 Role Permission Error]

end

subgraph "Custom Error Responses"

ValidationException --> FormatValidationError[Format Error Response
with Field Details]

ServerException --> LogException[Log Exception
to api_performance.log]

NotFoundException --> CustomNotFoundMessage[Return Custom
404 Message]

PermissionException --> CustomPermissionMessage[Return Custom
403 Message]

end

Error Handling Strategy

1. **Validation Errors:** Return 400 Bad Request with specific field validation errors
2. **Authentication Errors:** Return 401 Unauthorized for invalid/missing credentials
3. **Permission Errors:** Return 403 Forbidden for insufficient permissions
4. **Not Found Errors:** Return 404 Not Found for missing resources
5. **Server Errors:** Return 500 Internal Server Error and log details

Performance Monitoring

- The `PerformanceMiddleware` tracks request processing time
- Slow requests (>0.5s) are logged to `api_performance.log`
- Response headers include `X-Request-Duration` for client-side metrics

Access Control System

The API implements a sophisticated access control decision flow:

```
flowchart TD
    %% Main entry point
    Request([API Request]) --> AuthCheck{Is Authenticated?}

    %% Authentication check
    AuthCheck -->|No| Auth401[Return 401 Unauthorized]
    AuthCheck -->|Yes| EndpointType{Endpoint Type?}

    %% Different endpoint types
    EndpointType -->|Admin| AdminCheck{Is Admin?}
    EndpointType -->|Regular| RoleCheck{Role Check}
    EndpointType -->|Resource Owner| OwnerCheck{Is Owner?}
    EndpointType -->|Content Privacy| PrivacyCheck{Post Privacy?}

    %% Admin checks
    AdminCheck -->|Yes| AdminGranted[Access Granted]
    AdminCheck -->|No| Admin403[Return 403 Forbidden]

    %% Ownership checks
    OwnerCheck -->|Yes| OwnerGranted[Access Granted]
    OwnerCheck -->|No, but Read Request| ReadCheck{Is Read-Only Request?}
    OwnerCheck -->|No, and Write Request| Owner403[Return 403 Forbidden]

    %% Privacy checks
    PrivacyCheck -->|Public Post| PublicGranted[Access Granted]
    PrivacyCheck -->|Private Post| PrivateOwnerCheck{Is Post Owner?}

    PrivateOwnerCheck -->|Yes| PrivateOwnerGranted[Access Granted]
```

```
PrivateOwnerCheck -->|No, but Admin/Mod| RoleForPrivate{Is Admin/Moderator?}
PrivateOwnerCheck -->|No| Private403[Return 403 Forbidden]
```

Access Control Components

1. Permission Classes:

- **IsAuthenticated**: Ensures user is logged in
- **IsAdminUser**: Restricts access to admin users
- **IsOwnerOrReadOnly**: Allows read access to all, modify only for owners
- **IsPostOwnerOrPublic**: Handles post privacy visibility

2. Middleware:

- **RoleMiddleware**: Enforces role-based restrictions (e.g., only admins can delete)
- **DisableCSRFMiddleware**: Handles CSRF protection during development

3. Special Case Handling:

- Delete operations: Admin-only restriction via middleware
- Private content: Owner/admin/moderator access only
- Modified content: Owner-only modification with admin override

Performance Considerations

The API includes several performance optimizations:

1. Caching Strategy:

- Feed data is cached with versioned keys
- Cache invalidation on content updates
- Batch processing for large operations

2. Database Optimizations:

- Strategic indexing on frequently queried fields
- **select_related** and **prefetch_related** for efficient joins
- Annotated fields for counts (likes, comments)

3. Request Throttling:

- Authentication endpoints: 5 requests/minute
- Regular users: 40 requests/minute
- Anonymous users: 20 requests/minute

4. Monitoring:

- Request duration logging for slow endpoints
- Performance tracking via middleware
- Detailed error logging for troubleshooting

This technical documentation provides a comprehensive overview of the Connectly API's architecture, design patterns, and implementation details. It serves as a guide for developers working with the system, highlighting the flow of operations, security mechanisms, and optimization techniques employed throughout the application.